

INTEGRACIÓN DE COMPETENCIAS II

I. ANTECEDENTES GENERALES

Competencia:	
Criterios de evaluación:	-
Tipo de evidencia / Evidencia:	

Integrantes: Catherine Gomez

Diogo Von Muhlenbrock

Fabian Cares

Sebastián Pérez

II. INSTRUCCIONES Y FORMALIDADES

- **Tipo de actividad:** Individual
- **Contexto:** Laboratorio de Informática

III. DESARROLLO DE LA EVALUACIÓN

En la semana 6 las y los estudiantes deben presentar su software operativo, es decir funcionado, a su docente y al resto del curso, validando el cumplimiento de cada uno de los requerimientos solicitados por la empresa del caso. Una vez terminada la demostración deberán cargar en su Aula Virtual la evidencia de cada una de las etapas elaboradas desde la actividad 1.2 a 1.5. Los documentos que debe incluir esta entrega son:

1. Prototipo de Software (MVP)
 - a. Script de cada uno de los requerimientos.
2. Script de la base de datos
3. Plan de pruebas unitarias
4. Presentación de PPT del proyecto.

Link GitHub con Programa: <https://github.com/Bksp/AgroTech>

1. Prototipo de Software (MVP)

Script RF-01: Registrar parcelas y cultivos.

```
using System;
using System.Collections.Generic;
using System.Linq;
using AgroTech.DAL;
using AgroTech.DAL.Models;
using Microsoft.EntityFrameworkCore;

namespace AgroTech.BLL
{
    public class ParcelaService
    {
        private readonly Func<AgroTechDbContext> _contextFactory;

        public ParcelaService() : this({} => new AgroTechDbContext()) { }

        public ParcelaService(Func<AgroTechDbContext> contextFactory)
        {
            _contextFactory = contextFactory;
        }

        public Parcela RegistrarNuevaParcela(string nombre, string ubicacion, decimal dimensionesM2, int idUsuarioRegistrador)
        {
            if (string.IsNullOrEmpty(nombre) || string.IsNullOrEmpty(ubicacion))
                throw new ArgumentException("El nombre y la ubicación de la parcela son obligatorios.");

            var ubicacionNormalizada = ubicacion.ToLowerInvariant();
            if (!ubicacionNormalizada.Contains("maule") && !ubicacionNormalizada.Contains("Ñuble") && !ubicacionNormalizada.Contains("Ñuble"))
                throw new ArgumentException("La ubicación geográfica está restringida a Maule o Ñuble.");

            if (dimensionesM2 < 1.00m)
                throw new ArgumentException("Las dimensiones de la parcela deben ser mayores o iguales a 1.00 m².");

            var parcela = new Parcela
            {
                Nombre = nombre.Trim(),
                Ubicacion = ubicacion.Trim(),
                DimensionesM2 = dimensionesM2,
                RegistradoPor = idUsuarioRegistrador,
                FechaRegistro = DateTime.Now
            };

            using var db = _contextFactory();
            db.Parcelas.Add(parcela);
            db.SaveChanges();
            return parcela;
        }

        public List<Parcela> ObtenerTodas()
        {
            using var db = _contextFactory();
            return db.Parcelas.AsNoTracking().OrderByDescending(p => p.FechaRegistro).ToList();
        }
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using AgroTech.DAL;
using AgroTech.DAL.Models;
using Microsoft.EntityFrameworkCore;

namespace AgroTech.BLL
{
    public class CultivoService
    {
        private static readonly string[] EstadosValidos = { "Activo", "Cosechado", "Perdido por Plaga" };
        private readonly Func<AgroTechDbContext> _contextFactory;

        public CultivoService() : this(() => new AgroTechDbContext()) { }

        public CultivoService(Func<AgroTechDbContext> contextFactory)
        {
            _contextFactory = contextFactory;
        }

        public Cultivo RegistrarNuevoCultivo(int idParcela, string tipoCultivo, DateTime fechaSiembra, DateTime fechaEstimadaCosecha, int idUsuarioRegistrador)
        {
            if (idParcela <= 0)
                throw new ArgumentException("Debe seleccionar una parcela válida para asociar el cultivo.");

            if (string.IsNullOrEmpty(tipoCultivo))
                throw new ArgumentException("Debe indicar el tipo de cultivo.");

            if (fechaEstimadaCosecha <= fechaSiembra)
                throw new ArgumentException("La fecha estimada de cosecha debe ser posterior a la fecha de siembra.");

            using var db = _contextFactory();

            var existeParcela = db.Parcelas.Any(p => p.IdParcela == idParcela);
            if (!existeParcela)
                throw new ArgumentException("La parcela seleccionada no existe.");

            var cultivo = new Cultivo
            {
                IdParcela = idParcela,
                TipoCultivo = tipoCultivo.Trim(),
                FechaSiembra = fechaSiembra.Date,
                FechaEstimadaCosecha = fechaEstimadaCosecha.Date,
                Estado = "Activo",
                RegistradoPor = idUsuarioRegistrador,
                FechaRegistro = DateTime.Now
            };

            db.Cultivos.Add(cultivo);
            db.SaveChanges();
            return cultivo;
        }

        public void ActualizarEstado(int idCultivo, string nuevoEstado)
        {
            if (!EstadosValidos.Contains(nuevoEstado))
                throw new ArgumentException("Estado de cultivo inválido.");

            using var db = _contextFactory();
            var cultivo = db.Cultivos.Find(idCultivo);
            ?? throw new ArgumentException("El cultivo indicado no existe.");

            cultivo.Estado = nuevoEstado;
            db.SaveChanges();
        }

        public List<Cultivo> ObtenerTodos()
        {
            using var db = _contextFactory();
            return db.Cultivos
                .Include(c => c.Parcela)
                .AsNoTracking()
                .OrderByDescending(c => c.FechaRegistro)
                .ToList();
        }

        public void ActualizarCultivo(int idCultivo, DateTime fechaSiembra, DateTime fechaEstimadaCosecha, string estado)
        {
            if (fechaEstimadaCosecha <= fechaSiembra)
                throw new ArgumentException("La fecha estimada de cosecha debe ser posterior a la fecha de siembra.");
        }
    }
}
```



```
if (!EstadosValidos.Contains(estado))  
    throw new ArgumentException("Estado de cultivo inválido.");  
  
using var db = _contextFactory();  
var cultivo = db.Cultivos.Find(idCultivo)  
    ?? throw new ArgumentException("El cultivo indicado no existe.");  
  
cultivo.FechaSiembra = fechaSiembra.Date;  
cultivo.FechaEstimadaCosecha = fechaEstimadaCosecha.Date;  
cultivo.Estado = estado;  
  
db.SaveChanges();  
}  
  
public void EliminarCultivo(int idCultivo)  
{  
    using var db = _contextFactory();  
    var cultivo = db.Cultivos.Find(idCultivo)  
        ?? throw new ArgumentException("El cultivo indicado no existe.");  
  
    db.Cultivos.Remove(cultivo);  
    db.SaveChanges();  
}  
}
```

Script RF-03: Gestión de usuarios del sistema.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.RegularExpressions;
using AgroTech.DAL;
using AgroTech.DAL.Models;
using Microsoft.EntityFrameworkCore;

namespace AgroTech.BLL
{
    public class UsuarioService
    {
        private static readonly Regex CorreoInstitucional =
            new(@"^[A-Za-z0-9+_.-]+@agrotech\.cl$", RegexOptions.Compiled);
        private readonly Func<AgroTechDbContext> _contextFactory;

        public UsuarioService() : this(() => new AgroTechDbContext()) { }

        public UsuarioService(Func<AgroTechDbContext> contextFactory)
        {
            _contextFactory = contextFactory;
        }

        public List<Rol> ObtenerRoles()
        {
            using var db = _contextFactory();
            return db.Roles.AsNoTracking().OrderBy(r => r.IdRol).ToList();
        }

        public List<Usuario> ObtenerTodos()
        {
            using var db = _contextFactory();
            return db.Usuarios.Include(u => u.Rol).AsNoTracking().OrderBy(u => u.NombreCompleto).ToList();
        }

        public Usuario RegistrarUsuario(string nombreCompleto, string correo, string passwordTemporal, int idRol)
        {
            ValidarDatosBasicos(nombreCompleto, correo, idRol);

            if (string.IsNullOrEmpty(passwordTemporal) || passwordTemporal.Length < 6)
                throw new ArgumentException("La contraseña temporal debe tener al menos 6 caracteres.");

            using var db = _contextFactory();
```

```
        correo = correo.Trim();
        if (db.Usuarios.Any(u => u.CorreoElectronico == correo))
            throw new ArgumentException("Ya existe un usuario registrado con ese correo.");

        if (!db.Roles.Any(r => r.IdRol == idRol))
            throw new ArgumentException("El rol seleccionado no es válido.");

        var usuario = new Usuario
        {
            NombreCompleto = nombreCompleto.Trim(),
            CorreoElectronico = correo,
            PasswordHash = BCrypt.Net.BCrypt.HashPassword(passwordTemporal),
            IdRol = idRol,
            Estado = "Activo",
            FechaCreacion = DateTime.Now
        };

        db.Usuarios.Add(usuario);
        db.SaveChanges();
        return usuario;
    }

    public void ModificarUsuario(int idUsuario, string nombreCompleto, string correo, int idRol, string? nuevaPasswordOpcional)
    {
        ValidarDatosBasicos(nombreCompleto, correo, idRol);

        using var db = _contextFactory();
        var usuario = db.Usuarios.Find(idUsuario)
            ?? throw new ArgumentException("El usuario indicado no existe.");

        correo = correo.Trim();
        if (db.Usuarios.Any(u => u.CorreoElectronico == correo && u.IdUsuario != idUsuario))
            throw new ArgumentException("Ya existe otro usuario registrado con ese correo.");

        if (!db.Roles.Any(r => r.IdRol == idRol))
            throw new ArgumentException("El rol seleccionado no es válido.");

        usuario.NombreCompleto = nombreCompleto.Trim();
```

```

        usuario.CorreoElectronico = correo;
        usuario.IdRol = idRol;

        if (!string.IsNullOrEmpty(nuevaPasswordOpcional))
        {
            if (nuevaPasswordOpcional.Length < 6)
                throw new ArgumentException("La nueva contraseña debe tener al menos 6 caracteres.");
            usuario.PasswordHash = BCrypt.Net.BCrypt.HashPassword(nuevaPasswordOpcional);
        }

        db.SaveChanges();
    }

    public void SuspenderUsuario(int idUsuario, int idUsuarioQueEjecutaLaAccion)
    {
        if (idUsuario == idUsuarioQueEjecutaLaAccion)
            throw new InvalidOperationException("No puedes suspender tu propia cuenta mientras tienes la sesión activ
a.");

        using var db = _contextFactory();
        var usuario = db.Usuarios.Find(idUsuario);
        ?? throw new ArgumentException("El usuario indicado no existe.");

        usuario.Estado = "Suspendido";
        db.SaveChanges();
    }

    public void ReactivarUsuario(int idUsuario)
    {
        using var db = _contextFactory();
        var usuario = db.Usuarios.Find(idUsuario);
        ?? throw new ArgumentException("El usuario indicado no existe.");

        usuario.Estado = "Activo";
        db.SaveChanges();
    }

    public void ReactivarUsuario(int idUsuario)
    {
        using var db = _contextFactory();
        var usuario = db.Usuarios.Find(idUsuario);
        ?? throw new ArgumentException("El usuario indicado no existe.");

        usuario.Estado = "Activo";
        db.SaveChanges();
    }

    private static void ValidarDatosBasicos(string nombreCompleto, string correo, int idRol)
    {
        if (string.IsNullOrEmpty(nombreCompleto))
            throw new ArgumentException("El nombre completo es obligatorio.");

        if (string.IsNullOrEmpty(correo) || !CorreoInstitucional.IsMatch(correo.Trim()))
            throw new ArgumentException("El correo debe tener el formato institucional @agrotech.cl.");

        if (idRol <= 0)
            throw new ArgumentException("Debe seleccionar un rol.");
    }
}

```

Script RF-05: Realización de pruebas unitarias.

```
Code Blame 28 lines (23 loc) · 935 Bytes

1  <Project Sdk="Microsoft.NET.Sdk">
2
3    <PropertyGroup>
4      <TargetFramework>net10.0</TargetFramework>
5      <ImplicitUsings>enable</ImplicitUsings>
6      <Nullable>enable</Nullable>
7      <IsPackable>false</IsPackable>
8    </PropertyGroup>
9
10   <ItemGroup>
11     <PackageReference Include="coverlet.collector" Version="6.0.4" />
12     <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.12" />
13     <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
14     <PackageReference Include="Moq" Version="4.20.72" />
15     <PackageReference Include="xunit" Version="2.9.3" />
16     <PackageReference Include="xunit.runner.visualstudio" Version="3.1.4" />
17   </ItemGroup>
18
19   <ItemGroup>
20     <Using Include="Xunit" />
21   </ItemGroup>
22
23   <ItemGroup>
24     <ProjectReference Include="..\AgroTech.BLL\AgroTech.BLL.csproj" />
25     <ProjectReference Include="..\AgroTech.DAL\AgroTech.DAL.csproj" />
26   </ItemGroup>
27
28 </Project>
```

```
using System;
using System.Linq;
using AgroTech.BLL;
using AgroTech.DAL;
using AgroTech.DAL.Models;
using Microsoft.EntityFrameworkCore;
using Xunit;

namespace AgroTech.Tests
{
    public class CultivoServiceTests
    {
        private AgroTechDbContext GetInMemoryContext(string dbName)
        {
            var options = new DbContextOptionsBuilder<AgroTechDbContext>()
                .UseInMemoryDatabase(databaseName: dbName)
                .Options;

            var context = new AgroTechDbContext(options);
            context.Database.EnsureCreated();
            return context;
        }

        [Fact]
        public void RegistrarNuevoCultivo_Valido_GuardaCultivoEnBaseDeDatos()
        {
            // Arrange
            var dbName = Guid.NewGuid().ToString();
            using (var context = GetInMemoryContext(dbName))
            {
                var usuario = new Usuario { NombreCompleto = "Test", CorreoElectronico = "test@agrotech.cl", PasswordHash =
"hash" };
                context.Usuarios.Add(usuario);
                context.SaveChanges();

                var parcela = new Parcela { Nombre = "Parcela Test", Ubicacion = "Maule", DimensionesM2 = 1000, RegistradoPor = usuario.IdUsuario };
                context.Parcelas.Add(parcela);
                context.SaveChanges();
            }
        }
    }
}
```

```
// Act
using (var context = GetInMemoryContext(dbName))
{
    var service = new CultivoService(() => GetInMemoryContext(dbName));

    var fechaSiembra = DateTime.Now;
    var fechaCosecha = DateTime.Now.AddMonths(3);

    var cultivo = service.RegistrarNuevoCultivo(1, "Tomate", fechaSiembra, fechaCosecha, 1);

    Assert.NotNull(cultivo);
    Assert.Equal(1, cultivo.IdCultivo);
    Assert.Equal("Tomate", cultivo.TipoCultivo);
    Assert.Equal("Activo", cultivo.Estado);
}

// Assert
using (var context = GetInMemoryContext(dbName))
{
    var cultivosDb = context.Cultivos.ToList();
    Assert.Single(cultivosDb);
    Assert.Equal("Tomate", cultivosDb.First().TipoCultivo);
}
}

[Fact]
public void ActualizarCultivo_Valido_CambiaDatos()
{
    // Arrange
    var dbName = Guid.NewGuid().ToString();
    using (var context = GetInMemoryContext(dbName))
    {
        var usuario = new Usuario { NombreCompleto = "Test", CorreoElectronico = "test@agrotech.cl", PasswordHash =
"hash" };
        context.Usuarios.Add(usuario);
        var parcela = new Parcela { Nombre = "Parcela Test", Ubicacion = "Maule", DimensionesM2 = 1000, RegistradoPor
r = 1 };
        context.Parcelas.Add(parcela);
        var cultivo = new Cultivo { IdParcela = 1, TipoCultivo = "Tomate", Estado = "Activo", RegistradoPor = 1, FechaSiembra = DateTime.Now, FechaEstimadaCosecha = DateTime.Now.AddMonths(2) };
        context.Cultivos.Add(cultivo);
        context.SaveChanges();
    }
}
```

```

    }

    // Act
    var service = new CultivoService(() => GetInMemoryContext(dbName));
    var nuevaSiembra = DateTime.Now.AddDays(1);
    var nuevaCosecha = DateTime.Now.AddMonths(4);
    service.ActualizarCultivo(1, nuevaSiembra, nuevaCosecha, "Cosechado");

    // Assert
    using (var context = GetInMemoryContext(dbName))
    {
        var cultivoActualizado = context.Cultivos.Find(1);
        Assert.NotNull(cultivoActualizado);
        Assert.Equal("Cosechado", cultivoActualizado.Estado);
        Assert.Equal(nuevaSiembra.Date, cultivoActualizado.FechaSiembra);
        Assert.Equal(nuevaCosecha.Date, cultivoActualizado.FechaEstimadaCosecha);
    }
}

[Fact]
public void EliminarCultivo_Existente_BorraRegistro()
{
    // Arrange
    var dbName = Guid.NewGuid().ToString();
    using (var context = GetInMemoryContext(dbName))
    {
        var cultivo = new Cultivo { IdParcela = 1, TipoCultivo = "Tomate", Estado = "Activo", RegistradoPor = 1, FechaSiembra = DateTime.Now, FechaEstimadaCosecha = DateTime.Now.AddMonths(2) };
        context.Cultivos.Add(cultivo);
        context.SaveChanges();
    }

    // Act
    var service = new CultivoService(() => GetInMemoryContext(dbName));
    service.EliminarCultivo(1);

    // Assert
    using (var context = GetInMemoryContext(dbName))
    {
        var count = context.Cultivos.Count();
        Assert.Equal(0, count);
    }
}
}
}

```

2. Prototipo de Software (MVP)

```
CREATE DATABASE IF NOT EXISTS agrotech_db;
```

```
USE agrotech_db;
```

-- Tabla: Roles (Soporte para el Control de Acceso Basado en Roles)

```
CREATE TABLE Roles (
```

```
    id_rol INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nombre_rol VARCHAR(50) NOT NULL UNIQUE,
```

```
    descripcion VARCHAR(255),
```

```
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

-- Tabla: Usuarios (Credenciales de Autenticación, Trazabilidad y Datos Básicos)

```
CREATE TABLE Usuarios (
```

```
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
```

```
    id_rol INT NOT NULL,
```

```
    nombre_completo VARCHAR(150) NOT NULL,
```

```
    correo_electronico VARCHAR(150) NOT NULL UNIQUE,
```

```
    password_hash VARCHAR(255) NOT NULL, -- Contraseña hasheada (p. ej. bcrypt/argon2 desde la aplicación)
```

```
    estado VARCHAR(20) DEFAULT 'Activo', -- Estados: Activo, Suspendido
```

```
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    CONSTRAINT fk_usuario_rol
```

```
        FOREIGN KEY (id_rol)
```

```
        REFERENCES Roles(id_rol)
```

```
        ON DELETE RESTRICT
```

```
        ON UPDATE CASCADE
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

-- Tabla: Parcelas (Invernaderos o Sectores Físicos restringidos a Maule y Ñuble)

CREATE TABLE Parcelas (

id_parcela INT AUTO_INCREMENT PRIMARY KEY,

nombre VARCHAR(100) NOT NULL,

ubicacion VARCHAR(255) NOT NULL, -- Limitado a Maule y Ñuble

dimensiones_m2 DECIMAL(12,2) NOT NULL, -- Cambio de unidad de hectáreas a m2 (según RF-01 UI)

registrado_por INT NOT NULL, -- Llave foránea de trazabilidad operacional

fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT fk_parcela_usuario

FOREIGN KEY (registrado_por)

REFERENCES Usuarios(id_usuario)

ON DELETE RESTRICT

ON UPDATE CASCADE,

-- Candado a nivel de BD: impide el registro de dimensiones inferiores a 1 m2 (RF-01)

CONSTRAINT chk_dimensiones_m2

CHECK (dimensiones_m2 >= 1.00)

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Tabla: Cultivos (Ciclo biológico asociado de forma relacional a una parcela)

CREATE TABLE Cultivos (

id_cultivo INT AUTO_INCREMENT PRIMARY KEY,

id_parcela INT NOT NULL,

tipo_cultivo VARCHAR(100) NOT NULL, -- P. ej. "Tomate Cherry", "Arándano O'Neal"

fecha_siembra DATE NOT NULL,

fecha_estimada_cosecha DATE NOT NULL,

estado VARCHAR(50) DEFAULT 'Activo', -- Estados lógicos: Activo, Cosechado, Perdido por Plaga

registrado_por INT NOT NULL, -- Llave foránea de trazabilidad operacional

fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT fk_cultivo_parcela

```
FOREIGN KEY (id_parcela)
REFERENCES Parcelas(id_parcela)
-- ON DELETE RESTRICT bloquea la eliminación de una Parcela si contiene registros de cultivos
asociados
ON DELETE RESTRICT
ON UPDATE CASCADE,
CONSTRAINT fk_cultivo_usuario
FOREIGN KEY (registrado_por)
REFERENCES Usuarios(id_usuario)
ON DELETE RESTRICT
ON UPDATE CASCADE,
-- Candado a nivel de BD: asegura consistencia lógica de fechas agrícolas
CONSTRAINT chk_fechas_cultivo
CHECK (fecha_estimada_cosecha > fecha_siembra)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE INDEX idx_usuario_email ON Usuarios(correo_electronico);
CREATE INDEX idx_parcela_ubicacion ON Parcelas(ubicacion);
CREATE INDEX idx_cultivo_fechas ON Cultivos(fecha_siembra, fecha_estimada_cosecha);
CREATE INDEX idx_cultivo_estado ON Cultivos(estado);

-- Insertar Roles del Sistema (Soporte RBAC)
INSERT INTO Roles (nombre_rol, descripcion) VALUES
('Administrador', 'Control total del sistema, administración de usuarios y configuraciones.'),
('Supervisor', 'Gestión operativa, registro de cultivos, parcelas y monitoreo de producción.'),
('Trabajador', 'Lectura de tareas agrícolas y registro básico de datos diarios.');
```



```
-- Insertar Usuarios Semilla (Simulación de personal técnico de AgroTech)
INSERT INTO Usuarios (id_rol, nombre_completo, correo_electronico, password_hash, estado) VALUES
```

```
(1, 'Alejandro Valenzuela', 'a.valenzuela@agrotech.cl',  
'$2a$12$LMYS6fB46MvO85L.XQ6Vee.n9L.UfLpB3ZqWjK3jFjYF89YhH29Y6', 'Activo'),  
(2, 'María José Fuentes', 'm.fuentes@agrotech.cl',  
'$2a$12$KPhS7gC57NvP96M.YR7Wff.o0M.VgMqC4aRXk4kGkZG80Zii30Z7', 'Activo'),  
(3, 'Pedro Carrasco', 'p.carrasco@agrotech.cl',  
'$2a$12$JNiR8hD68OvQ07N.ZS8Xgg.p1N.WhNrD5bSYI5IHIAH91Ajj41A8', 'Activo');
```

-- Insertar Parcelas de Demostración (Ubicaciones restringidas a Maule y Ñuble, dimensiones en m2)

-- Corrección del campo 'ubicación' por 'ubicacion' para evitar errores de sintaxis

INSERT INTO Parcelas (nombre, ubicacion, dimensiones_m2, registrado_por) VALUES

('Invernadero Central Norte', 'Maule - Fundo Los Alerces', 25000.00, 2),

('Sector Sur Lote B', 'Ñuble - Fundo San Francisco', 48000.00, 2),

('Invernadero Hidropónico A', 'Maule - Sector Central', 12000.00, 1);

-- Insertar Cultivos de Demostración asociados a las parcelas anteriores

INSERT INTO Cultivos (id_parcela, tipo_cultivo, fecha_siembra, fecha_estimada_cosecha, estado, registrado_por) VALUES

(1, 'Tomate Cherry', '2026-05-15', '2026-09-15', 'Activo', 2),

(1, 'Lechuga Hidropónica', '2026-08-01', '2026-09-30', 'Activo', 2),

(2, 'Arándano O Neal', '2025-06-10', '2026-11-20', 'Activo', 2),

(3, 'Frutilla Albión', '2026-04-01', '2026-08-15', 'Cosechado', 1);

-- Consulta relacional unificada con datos de trazabilidad e integridad de parcela y cultivo

SELECT

c.id_cultivo,

p.nombre AS nombre_parcela,

p.ubicacion AS ubicacion_parcela,

p.dimensiones_m2,

c.tipo_cultivo,

c.fecha_siembra,

c.fecha_estimada_cosecha,



```
c.estado AS estado_cultivo,  
u.nombre_completo AS registrado_por_usuario,  
r.nombre_rol AS rol_del_registrador  
FROM Cultivos c  
INNER JOIN Parcelas p ON c.id_parcela = p.id_parcela  
INNER JOIN Usuarios u ON c.registrado_por = u.id_usuario  
INNER JOIN Roles r ON u.id_rol = r.id_rol  
ORDER BY c.fecha_siembra DESC;
```



3. Presentación.

https://prezi.com/view/qTbaSpoe6u1JORA5g566/?referral_token=CxwGs9lnB3FN